

LedgerSync

Product and Engineering Report

A trust-first internal money-transfer and ledger platform for fintech and vertical-SaaS teams.

The product in one sentence

LedgerSync gives teams an exact, auditable, retry-safe financial ledger and transfer core before they take on the complexity of payment rails.

Primary buyer	Fintech and vertical-SaaS product teams that need account balances, internal transfers, and an explainable ledger.
Daily users	Integration engineers, finance/operations teams, and authorized account holders using the dashboard.
Launch surface	API-first financial core with an operator dashboard and BFF. A public marketing site is separate and non-financial.
Initial scope	Closed-loop, same-currency transfers between LedgerSync accounts. Bank rails, FX, custody, chargebacks, and external settlement are intentionally deferred.

Report scope: implemented foundation, planned product capabilities, architecture direction, and the high-fidelity transfer-detail wireframe.

1. Product idea and the foundation being replaced

The commercial wedge

Start with teams building wallets, credits, internal treasury, marketplace balances, or other ledger-backed products. Their immediate need is trustworthy financial state, not a full banking stack.

LedgerSync is deliberately positioned as financial infrastructure. It provides the dependable accounting and transfer layer that products need before integrating external payment rails. The operator dashboard makes the system inspectable for finance and support teams; it is not merely a consumer wallet UI.

Previous prototype foundation: a learning/demo system with multiple loosely connected services. It contained invalid runtime paths, float-based money handling, browser calls to an incompatible transport boundary, mutable balances without a durable accounting trail, cache-first assumptions, and incomplete operational controls.

Previous state	Four demo-style services, incomplete build paths, direct Redis publication, raw gRPC/browser mismatch, public-facing diagnostics, and float/double values in money paths.
New foundation	A modular Go financial core, independent outbox worker, Next.js dashboard/BFF, PostgreSQL financial authority, Redis only as a versioned disposable read cache, and controlled migrations.
Business consequence	The product changes from a demo balance viewer into an auditable ledger platform that can support a controlled production pilot.

Non-negotiable rule: a financial balance must always be explainable from immutable ledger entries, never from a cache or an editable balance field alone.

2. New foundation implemented in the repository

The following are implemented foundations and contracts. They establish the safe base for the remaining transfer workflow, projection worker, and operator product work.

Exact money	Currency-qualified integer minor units, strict decimal parsing, supported precision policy, and rejection of float-like or unrepresentable amounts.
Ledger rules	Account status/ownership primitives, transfer states, positive posting values, debit-credit balancing validation, and append-only ledger migration protections.
Data model	Versioned PostgreSQL migrations for accounts, owners, balance projections, transfers, journals, postings, idempotency records, outbox events, and audit events.
Safe retries	Canonical request fingerprinting, idempotency-key validation, replay decision rules, mismatch conflicts, and in-progress request protection.
Private boundary	A Go API bootstrap, liveness/readiness endpoints, correlation IDs, controlled public-error envelope, OIDC principal abstraction, and a development-only identity adapter.
BFF security	Signed session design, CSRF/origin checks, security headers, bounded JSON input, and a rate-limit interface. Browser code does not own financial credentials.
Operations	Dockerfiles, private Compose topology, migration command, reconciliation command entry point, CI, OpenAPI contract validation, unit tests, and an integration-harness foundation.

Current implementation boundary

The repository now has a secure, testable foundation. The atomic PostgreSQL transfer repository, customer-facing transfer endpoints, outbox processing, cache projection, and complete dashboard journeys remain roadmap implementation work.

3. New product capabilities being added

These are the planned capabilities that turn the foundation into the first production pilot. They are intentionally sequenced so financial correctness is proven before feature breadth.

Safe transfer command	Authorized source-account debit, exact validation, deterministic account locking, no-overdraft concurrency protection, atomic journal/postings/projection/outbox commit, and a stable transfer receipt.
Read-your-writes balances	After a completed transfer, a user receives a minimum-version requirement. The dashboard reads cache only when it is new enough; otherwise it waits briefly and falls back to PostgreSQL.
Reliable event delivery	A transactional outbox records balance-change events with the transfer. The worker publishes at least once; consumers use event IDs and versions to remain idempotent.
Account and history views	Authorized account list, exact balances, transaction history, and a clear balance freshness state without exposing cache or consistency internals.
Operator confidence	Reconciliation, audit evidence, outbox monitoring, recovery playbooks, alerts, backup/restore exercises, and release evidence.
Accessible product UX	Transfer form, confirmation, processing/error/retry states, keyboard support, meaningful status messages, contrast, and responsive account views.

What is explicitly not in the MVP

External bank rails, FX, scheduled transfers, chargebacks, multi-region active-active writes, custom password auth, Kubernetes, and service-mesh complexity are deferred until the core proves itself.

4. Architecture and trust model

The architecture keeps the financial write boundary deliberately small. PostgreSQL is the permanent system of record. Redis improves speed but is never allowed to decide a financial truth.

Browser	Next.js dashboard + BFF	Go Transfer Core	PostgreSQL
HTTPS only same origin	Session, CSRF, input limits private API calls	Authorization, idempotency ledger transaction	Transfers, postings, projections outbox, audit records
		Outbox worker	Redis Streams/cache
		At-least-once relay retry and recovery	Versioned fast reads never financial authority

Trust rules:

- No floating-point money.
- No edits or deletes of posted ledger history; corrections are compensating movements.
- A matching idempotency retry returns the original result and never moves money again.
- Transfer, ledger, versions, response, and outbox either commit together or do not commit.
- Redis loss causes primary-database fallback and later cache rebuild, not loss of financial truth.

5. Operator and account-holder experience

The primary dashboard should communicate financial certainty in plain language. It must show what happened, the result, and any operational delay without asking users to understand queues, cache versions, or tokens.

1. Sign in	OIDC-backed BFF session; users receive only the accounts and actions they are allowed to access.
2. Choose accounts	The transfer form accepts an exact decimal text amount, validates currency and ownership, and generates one retry-safe idempotency key.
3. Confirm transfer	The user sees processing, invalid, insufficient-funds, completed, or safely-retried states in human language.
4. See proof	Confirmation displays transfer ID, source/destination, exact amount, and the resulting balances. The ledger view shows the balanced debit and credit entries.
5. Handle delay	If a webhook/event is delayed, the transfer remains posted and balanced. The UI distinguishes delivery delay from financial failure.

Design principle

Expose final financial state, not implementation noise. Users should see a successful transfer, clear balances, and a truthful temporary-unavailability message when a correct answer cannot be obtained.

6. High-fidelity UI wireframe - transfer detail

The generated high-fidelity wireframe below represents the target operator transfer-detail experience. It surfaces status, exact amounts, balanced immutable ledger entries, resulting balance versions, audit timeline, and an explicitly separate webhook-delivery delay.

LedgerSync

Overview

Transfers

Accounts

Ledger

Notifications

Reconciliations

Reports

Counterparties

Audit Log

Settings

Global Treasury

Ava Patel Operator

Search transfers, accounts, ledgers...

18 Aug 2026, 09:20 UTC

Transfers > TRF-20260818-000734

Transfer TRF-20260818-000734 POSTED

From

Global Treasury - Operating
LED-1001

To

Global Treasury - Payroll
LED-2002

Amount

USD 12,480.00

Committed

18 Aug 2026, 09:15 UTC

Client reference

PAYRUN-2026-WK33

Transfer ID

TRF-20260818-000734

Ledger entries are balanced

Total debits: USD 12,480.00 | Total credits: USD 12,480.00

Ledger entries (immutable)

#	Entry ID	Type	Account (Ledger)	Amount (USD)	Resulting balance (USD)	Version
1	ENT-20260818-001468	Debit	Operating Cash (LED-1001)	12,480.00	1,234,567.89	v42
2	ENT-20260818-001469	Credit	Payroll Cash (LED-2002)	12,480.00	567,890.12	v17

Currency: USD

Post-transfer balances

As of 18 Aug 2026, 09:15 UTC

Operating Cash (LED-1001)	Payroll Cash (LED-2002)
USD 1,234,567.89 (v42)	USD 567,890.12 (v17)

Audit timeline

Transfer created	Ava Patel (ap@globaltreasury.com)	18 Aug 2026, 09:14:02 UTC
Transfer validated	System	18 Aug 2026, 09:14:08 UTC
Ledger entries posted	System	18 Aug 2026, 09:15:00 UTC
Transfer committed	System	18 Aug 2026, 09:15:00 UTC

Webhook delivery delayed

DELATED

LedgerSync has attempted delivery to Payroll System via webhook, but delivery is delayed.

This does not affect the posted status of this transfer. Ledger entries are final and balanced.

Endpoint

https://payroll.example.com/webhook

Last attempt

18 Aug 2026, 09:19:45 UTC

Next retry

18 Aug 2026, 09:24:45 UTC

Attempts

3

View delivery attempts

Webhook retries will continue automatically with exponential backoff.

Wireframe interpretation: posted transfer status and ledger balance remain authoritative even while downstream webhook delivery is delayed. That distinction is a core trust signal for operations teams.

7. Delivery roadmap and MVP definition

The product should not launch as a self-serve public payments system. The correct initial finish line is a controlled production pilot with a small number of design partners and strong correctness evidence.

Now - foundation	Runnable governed repository, exact money/ledger primitives, migrations, security boundary, contract, CI, and the wireframe direction.
Next - transaction core	Atomic PostgreSQL transfer repository, ownership lookup, account locks, balances/versions, idempotency response replay, posting/outbox persistence, and transfer API.
Then - truthful reads	Outbox worker, version-aware Redis projection, signed minimum-version requirement, bounded wait and primary fallback.
Pilot readiness	Dashboard journeys, access controls, reconciliation, alerts, secret management, backup/restore drills, fault tests, load tests, and release evidence.
Later - justified scale	Read replicas only for measured demand; durable broker or bank/payment-rail integrations only as separately specified programs.

Recommended MVP success gates

- 2-3 design-partner customers in a single jurisdiction and one initial currency.
- Meaningful pilot transfer activity with zero unexplained ledger-to-projection reconciliation mismatches.
- Proof that concurrent debits never overdraw and duplicate retries never create a second movement.
- Confirmed object-level authorization, audit evidence, recovery runbooks, and restore drill evidence.
- A measured transfer latency objective, starting with at least 95% of healthy requests receiving a final outcome within two seconds.

Strategic decision

Build the ledger and transfer engine first. Add a public marketing site to explain the product and build trust, but keep it isolated from the authenticated dashboard and financial API boundary.